Universidad Nacional de Costa Rica

Escuela de Informática

Sistema de Planificación Inteligente de Mantenimiento

Proyecto I

Curso: Programación II

1. Descripción General

El presente proyecto tiene como objetivo el desarrollo de un sistema en lenguaje C++ orientado a la planificación automática de mantenimiento para equipos de laboratorio académico.

A diferencia de sistemas centrados únicamente en almacenamiento de datos o en operaciones de registro, este proyecto se enfoca en la modelación del comportamiento dinámico del sistema, la toma de decisiones algorítmica y la simulación de escenarios operativos bajo restricciones reales.

El sistema deberá simular un periodo de 30 días de operación. Durante ese tiempo deberá evaluar continuamente el estado de los equipos, la aparición o acumulación de incidencias, el riesgo asociado a cada equipo y la disponibilidad limitada de recursos técnicos.

En otras palabras, el propósito principal del proyecto no es solo construir clases y guardar información, sino desarrollar una solución que **decida qué hacer, cuándo hacerlo y por qué hacerlo**, usando criterios programados formalmente.

2. Objetivo General

Diseñar e implementar un sistema capaz de modelar, priorizar, planificar y simular procesos de mantenimiento técnico, aplicando programación orientada a objetos, estructuras de datos, algoritmos y manejo robusto de errores en C++.

3. Objetivos Específicos

- Modelar correctamente el dominio mediante clases cohesionadas, relaciones apropiadas y contratos consistentes.
- Aplicar principios de diseño como SRP, OCP, LSP, KISS y YAGNI dentro de la solución propuesta.

- Implementar mecanismos de priorización dinámica basados en criterios cuantificables.
- Simular el comportamiento del sistema durante 30 días bajo condiciones de recursos limitados.
- Aplicar polimorfismo real mediante el uso de una clase base y clases derivadas.
- Implementar al menos un patrón de diseño de manera real y verificable dentro de la solución.
- Utilizar algoritmos propios de ordenamiento o priorización, justificando su uso.
- Gestionar errores mediante excepciones personalizadas y validación de entradas.

4. Problema a Resolver

Un laboratorio académico dispone de múltiples equipos con diferentes niveles de criticidad. Con el paso del tiempo, estos equipos se degradan, pueden fallar y presentar incidencias que afectan su funcionamiento.

Cada día, el laboratorio solo dispone de **3 técnicos** para atender tareas de mantenimiento. Como consecuencia, no todos los equipos pueden ser intervenidos al mismo tiempo, por lo que el sistema debe decidir automáticamente cuáles deben recibir atención primero.

El sistema debe tomar decisiones considerando al menos:

- La criticidad del equipo.
- La cantidad de incidencias activas.
- El tiempo que el equipo ha permanecido sin atención o inactivo.
- La cantidad limitada de recursos disponibles por día.

Importante: el problema principal no consiste en guardar registros, sino en **tomar decisiones inteligentes bajo restricciones**.

Durante cada día de la simulación, el sistema debe responder esta pregunta:

¿Cuáles son los 3 equipos que deben recibir mantenimiento hoy para reducir al máximo el riesgo operativo del laboratorio?

Para resolverlo, el programa deberá:

1. Evaluar el estado actual de todos los equipos.

2. Calcular una prioridad dinámica para cada uno.
3. Ordenar los equipos según esa prioridad.
4. Seleccionar los 3 más urgentes.
5. Ejecutar el mantenimiento correspondiente.
6. Actualizar el estado global del sistema.
7. Repetir este proceso durante 30 días consecutivos.

5. Requerimientos Funcionales

El sistema debe implementar obligatoriamente las siguientes funcionalidades:

- **RF1 - Registro inicial:** El sistema debe permitir cargar o registrar un conjunto inicial de equipos e incidencias.
- **RF2 - Simulación completa:** El sistema debe ejecutar una simulación de exactamente 30 días consecutivos.
- **RF3 - Degradación diaria:** Cada día, los equipos deben deteriorarse conforme a reglas explícitas definidas dentro del programa.
- **RF4 - Cálculo de prioridad:** La prioridad de atención de cada equipo debe calcularse utilizando la siguiente fórmula obligatoria:

```
prioridad = (criticidad * 0.5) +
            (incidencias_activas * 0.3) +
            (tiempo_inactivo * 0.2)
```

- **RF5 - Reordenamiento diario:** Todos los equipos deben ser reordenados cada día de acuerdo con su prioridad, de mayor a menor.
- **RF6 - Selección limitada por recursos:** El sistema solo puede asignar mantenimiento a un máximo de 3 equipos por día.
- **RF7 - Ejecución de mantenimiento:** El mantenimiento aplicado a un equipo debe modificar su estado, mejorar su condición o disminuir su nivel de riesgo, según las reglas implementadas por el estudiante.
- **RF8 - Actualización del sistema:** Después de cada día, el sistema debe recalcular las variables necesarias para la siguiente iteración.

- **RF9 - Generación de reportes:** El sistema debe producir reportes diarios y acumulados que incluyan, como mínimo:
  - Equipos atendidos
  - Prioridades calculadas
  - Equipos pendientes de atención
  - Riesgo global del laboratorio
  - Estado general de la simulación
- **RF10 - Persistencia en texto:** Los resultados de la simulación deben guardarse en archivos de texto para poder revisar posteriormente el comportamiento completo del sistema.

6. Requerimientos No Funcionales

- El código debe ser modular, mantenible y de bajo acoplamiento.
- La solución debe compilar en estándar C++17 o superior.
- El sistema debe mostrar mensajes de error claros y técnicamente correctos.
- Debe existir trazabilidad de las decisiones tomadas durante cada día de la simulación.
- El rendimiento debe ser adecuado para conjuntos de prueba amplios.
- La estructura del código debe evidenciar buenas prácticas de diseño.

7. Restricciones Técnicas Obligatorias

La solución debe cumplir obligatoriamente con las siguientes restricciones técnicas:

- El lenguaje de implementación será **C++**.
- No se permite utilizar librerías externas para resolver la parte de priorización o planificación.
- Debe existir al menos:
  - 1 clase abstracta con comportamiento común,
  - 2 clases derivadas concretas,
  - 1 colección polimórfica que trabaje con la clase base.

- Debe implementarse **uno** de los siguientes patrones de diseño:
  - Strategy
  - Template Method
  - Decorator
- El patrón debe usarse de forma **real** dentro de la simulación. No basta con crear clases vacías o decorativas.
- Debe implementarse al menos un algoritmo propio, por ejemplo:
  - Heap manual
  - Merge Sort
  - QuickSort
- Debe utilizarse una búsqueda optimizada, por ejemplo:
  - Búsqueda binaria sobre colección ordenada
  - Hash simple implementado por el estudiante
- Deben utilizarse excepciones personalizadas para errores como:
  - Archivo inválido
  - Formato inválido
  - Operación inconsistente
- Debe existir un caso de dependencia mutua entre dos clases del dominio, resuelto correctamente mediante `forward declaration`.

8. Aplicación Obligatoria de Polimorfismo y Relaciones Técnicas

8.1. Upcasting

La solución debe utilizar objetos derivados a través de punteros o referencias a una clase base. Esto significa que el sistema debe trabajar polimórficamente con diferentes tipos de mantenimiento o comportamiento especializado sin depender directamente de la clase concreta en todo momento.

8.2. Downcasting Seguro

Debe existir al menos un caso donde se necesite acceder a comportamiento exclusivo de una clase derivada. En ese caso, el downcasting debe realizarse mediante `dynamic_cast` y con validación explícita.

No se permite el uso de `static_cast` para downcasting.

8.3. Dependencia Mutua Controlada

Debe plantearse una situación donde dos clases se relacionen mutuamente y dicha relación sea resuelta técnicamente de forma correcta mediante archivos separados, `forward declaration` e implementación apropiada en archivos fuente.

9. Simulación Operativa

La simulación constituye el núcleo del proyecto.

El sistema debe ejecutar un ciclo de 30 iteraciones, donde cada iteración representa un día de operación. En cada día debe ocurrir, como mínimo, la siguiente secuencia:

1. **Degradación del sistema:** todos los equipos pierden estado o aumentan su nivel de desgaste.
2. **Actualización de incidencias:** pueden aparecer nuevas incidencias o mantenerse las ya existentes.
3. **Cálculo de prioridad:** se calcula la prioridad de cada equipo usando la fórmula definida.
4. **Ordenamiento:** se ordenan los equipos desde el más urgente hasta el menos urgente.
5. **Selección:** se eligen únicamente los 3 equipos con mayor prioridad.
6. **Mantenimiento:** se aplica la lógica correspondiente a los equipos seleccionados.
7. **Actualización global:** se recalculan estados, backlog, riesgo y demás métricas.
8. **Registro del día:** se genera una salida que permita entender qué ocurrió durante ese día.

Nota importante: la simulación debe ser acumulativa. Es decir, lo que ocurre en un día debe afectar el comportamiento de los días siguientes.

10. Qué Debe Entenderse por una Solución Completa

Para evitar ambigüedades, una solución completa debe evidenciar claramente lo siguiente:

- Que existen equipos modelados como objetos con atributos y comportamiento.
- Que el sistema calcula prioridades dinámicamente y no de forma fija.
- Que la simulación realmente avanza día por día.
- Que existe una decisión automática sobre qué equipos atender.
- Que el uso de polimorfismo no es decorativo, sino parte del flujo del sistema.
- Que el algoritmo implementado tiene un propósito real dentro de la solución.
- Que el sistema produce evidencia verificable de funcionamiento.

11. Entregables

El estudiante debe entregar lo siguiente:

- **E1 - Código fuente completo:** Proyecto completo en C++ que compile y ejecute sin errores.
- **E2 - Documento técnico:** Documento donde se expliquen, al menos, los siguientes puntos:
  - Diseño de clases
  - Relaciones entre clases
  - Patrón de diseño implementado
  - Algoritmo utilizado
  - Decisiones técnicas relevantes
  - Manejo de errores
- **E3 - Datos de prueba:** Conjunto de prueba con al menos:
  - 100 equipos
  - 300 incidencias
- **E4 - Resultados de simulación:** Archivos de texto que permitan revisar el comportamiento de la simulación completa.

- **E5 - Evidencia de ejecución:** Capturas, salidas en consola o registros que demuestren que el sistema funciona correctamente.
- **E6 - Presentación técnica:** Exposición breve de 5 a 8 minutos donde el estudiante explique y defienda técnicamente su solución.

12. Rúbrica de Evaluación

12.1. Resumen General

Criterio	Peso	Descripción
Modelado orientado a objetos	25 %	Diseño de clases, relaciones, cohesión, polimorfismo y consistencia del dominio
Priorización algorítmica	30 %	Implementación del sistema de prioridad, ordenamiento y toma de decisiones
Persistencia y trazabilidad	20 %	Generación de salidas, registro de simulación y evidencia verificable
Integración y simulación	15 %	Funcionamiento completo y acumulativo de los 30 días
Calidad técnica	10 %	Manejo de errores, claridad del código y buenas prácticas

12.2. Escala de desempeño

- **Excelente:** cumple completamente, con diseño sólido, solución correcta y dominio técnico claro.
- **Bueno:** cumple adecuadamente, aunque presenta detalles menores de diseño o implementación.
- **Regular:** cumple parcialmente, con debilidades visibles o cobertura incompleta.
- **Deficiente:** no cumple los requisitos esenciales o la solución es inconsistente.

13. Criterios de Evaluación Detallados

13.1. 1. Modelado orientado a objetos (25 puntos)

Se evaluará:

- Calidad del diseño de clases.

- Cohesión y responsabilidad de cada clase.
- Relaciones entre objetos.
- Uso correcto de herencia y polimorfismo.
- Consistencia general del dominio.

13.2. 2. Priorización algorítmica (30 puntos)

Se evaluará:

- Correcta aplicación de la fórmula de prioridad.
- Reordenamiento diario de los equipos.
- Implementación del algoritmo propio.
- Justificación técnica del algoritmo seleccionado.
- Capacidad del sistema para decidir automáticamente qué atender.

13.3. 3. Persistencia y trazabilidad (20 puntos)

Se evaluará:

- Claridad de los reportes generados.
- Registro completo de la simulación.
- Capacidad de revisar qué ocurrió en cada día.
- Organización y legibilidad de la salida producida.

13.4. 4. Integración y simulación (15 puntos)

Se evaluará:

- Funcionamiento estable del flujo general.
- Correcta ejecución de los 30 días.
- Coherencia entre estado inicial, cambios diarios y estado final.
- Integración real de todos los módulos del sistema.

13.5. 5. Calidad técnica (10 puntos)

Se evaluará:

- Manejo robusto de errores.
- Validación de entradas.
- Claridad del código.
- Organización del proyecto.
- Uso adecuado de buenas prácticas.

14. Reglas de Incumplimiento Crítico

Se considerarán faltas graves las siguientes situaciones:

- El proyecto no compila.
- No existe simulación real de 30 días.
- No existe un algoritmo funcional de priorización u ordenamiento.
- No se demuestra polimorfismo real.
- No se implementa el patrón de diseño obligatorio.
- No existe evidencia verificable de ejecución.
- El proyecto se limita únicamente a registrar datos sin tomar decisiones automáticas.

Estas faltas impactarán directamente la calificación final del proyecto.

15. Ejemplo Referencial de Entrada y Salida

15.1. Ejemplo de entrada

```
EQ-001;criticidad=9;estado=70
EQ-002;criticidad=6;estado=82
INC;EQ-001;severidad=ALTA;dia=3
INC;EQ-002;severidad=MEDIA;dia=5
```

15.2. Ejemplo de salida esperada

```
Dia 12
Top prioridad: EQ-001 (8.7), EQ-004 (8.1), EQ-010 (7.9)
Asignados: EQ-001, EQ-004, EQ-010
Backlog pendiente: 17
Riesgo global: ALTO
```

16. Criterio Global de Aprobación

- Puntaje total del proyecto: **100 puntos**.
- Para aprobar, no basta con obtener puntos parciales: también debe evidenciarse cumplimiento técnico esencial del proyecto.